KING'S College LONDON

Faculty of Natural, Mathematical, and Engineering Sciences

Department of Informatics

Software Engineering Group Project

StudyFlax: Time Management for Higher Education (Academic Project)

Team Flax-10

Aishah Ahmed

Eric Almeida

Zerun (Harry) Chen

Cate Tay (Cate) Chew

Indixora Jozal

Kausar Khan

Hamed Latif

Dian Mahmood

Binuli Nevanma Markawitage

March 29, 2026

Contents

1 Introduction **1**

2 Product **2**
2.1 Objectives, context and key stakeholders . . . 2
2.2 Requirements and specifications . . . 3

3 Design **5**
3.1 System Overview . . . 5
3.2 Architectural Pattern . . . 5
3.3 Frontend Design . . . 8
3.4 Backend Design . . . 8
3.4.1 API Layer . . . 8
3.4.2 Models . . . 9
3.4.3 Serialisers . . . 9
3.4.4 Signals . . . 9
3.4.5 Authentication . . . 9
3.4.6 Admin . . . 9
3.5 UML Component Diagrams . . . 10
3.6 Entity Relationship Diagrams . . . 15
3.7 Key Design Decisions . . . 18
3.7.1 Custom User Model with Unified Role System . . . 18
3.7.2 Token-Based Authentication . . . 18
3.7.3 Course-Prefix Mapping for Module Onboarding . . . 18
3.7.4 CSS Variables for Theming . . . 18
3.7.5 Tasks as a Standalone Page . . . 19
3.7.6 Navigation: Top Bar and Bottom Bar . . . 19
3.7.7 Modular Bento Dashboard with Widget Customisation . . . 19
3.7.8 Auto-Scheduler Design . . . 19
3.8 External Integrations . . . 20

4 Testing **21**
4.1 Automated Testing . . . 21
4.2 Test Coverage . . . 22
4.3 Test Code Quality . . . 22
4.4 Manual Testing . . . 22
4.5 Limits of Testing . . . 23
4.6 Threats to Validity . . . 23

5 Project management **24**
5.1 Team Structure and Roles 24
5.2 Project Evolution 24
5.3 Communication and Collaboration 25
5.4 Software Quality and Standards 25
5.5 Reflection and Lessons Learned 25

Chapter 1

Introduction

Indixora Jozal

StudyFlax is a time management platform designed to support university students in balancing the many demands of higher education. It helps students at King's College London coordinate lectures, deadlines, coursework, society commitments, and personal goals within a single, unified system. Instead of becoming another task to manage, StudyFlax simplifies organisation by reducing cognitive load, allowing students to focus their energy on learning and personal growth. By providing a clear view of both academic and extracurricular responsibilities, the platform empowers users to plan effectively, make better use of their time, and maintain a balanced schedule. Teaching staff are also able to use the platform to create and manage modules, publish reading lists and assignment deadlines, and communicate key course information directly to enrolled students without relying on external systems.

StudyFlax is built as a Single Page Application accessible on both desktop and mobile. The frontend is developed with React and Vite, and the backend is a Django REST Framework application backed by a PostgreSQL database. The development environment is containerised using Nix Flakes and Docker Compose, providing a reproducible setup across the team.

Chapter 2

Product

Aishah Ahmed, Binuli Nevanma Markawitage

2.1 Objectives, context and key stakeholders

Higher education demands a level of autonomous organisation that many students find genuinely difficult to sustain. At KCL, students are expected to independently track deadlines, manage coursework across multiple modules, attend society commitments, and maintain a healthy work-life balance, all without a single system to support them. The administrative burden falls equally on teaching staff, who frequently field repeated queries about module details, reading lists, and submission dates that could be resolved by a well-designed information platform.

Flax-10 identified three primary stakeholder groups whose needs shaped the objectives of StudyFlax:

- **KCL Students**: the primary users, who stand to gain the most from reduced organisational burden and a clearer view of their academic and personal commitments.
- **KCL Teaching Staff**: who benefit from a direct channel to broadcast module information to students, reducing repetitive administrative communication.
- **KCL Society Organisers**: who gain a dedicated space to reach and engage their members without relying on external platforms.

These stakeholders informed three core objectives for StudyFlax. The first is **consolidation**: to give students a single platform through which all academic and extracurricular commitments are visible and actionable, eliminating the need to consult multiple disconnected tools. The second is **personalisation**: to provide productivity support that adapts to a student's individual habits and workload, rather than imposing a rigid, one-size-fits-all schedule. The third is **communication**: to close the information gap between teaching staff and students, ensuring that module content, deadlines, and announcements reach students directly within the tool they already use to manage their time.

Together, these objectives reflect a belief that the right digital infrastructure can meaningfully reduce the stress and cognitive overhead that student life imposes.

2.2 Requirements and specifications

The requirements for StudyFlax were derived directly from the three objectives above. The system that was delivered can be understood as a set of feature areas, each of which addresses one or more of the objectives of consolidation, personalisation, and communication.

Calendar and timetable integration. Students can access the Calendar page through the main navigation. This area of the system allows users to create personal events, manage recurring events, and import timetable data from KCL's iCalendar feed. Its primary contribution is consolidation: academic timetables, personal plans, and other commitments are brought into one view rather than being split across separate systems.

Task management. Students can access the Tasks page from the main navigation and can also interact with task data through the homepage dashboard. The delivered system supports task lists, task completion, prioritisation, time logging, and links between tasks and related academic work. This feature area supports consolidation by giving users a dedicated place to organise actionable work, and also supports personalisation by allowing students to structure their workload in the way that suits them best.

Focus timer and study tracking. The timer is accessible from the main navigation and from timer-related controls in the interface. It records structured study sessions and contributes to statistics such as streaks, study history, and weekly progress. This supports personalisation by helping students build routines around their own working habits rather than simply storing deadlines passively.

Auto-scheduler and dashboard customisation. Students can access the auto-scheduler from the scheduling interface and can personalise the homepage through widget configuration. The auto-scheduler uses deadlines and user settings, such as sleep schedule and preferred study time, to generate a revision plan. These features were specified to ensure the platform does not merely display information, but actively adapts to the student's context and preferences.

Modules and academic information. Teaching staff can access the Modules section to create and manage modules, deadlines, and reading lists, while students can view the same information in the student-facing interface. This feature area supports communication by providing a direct route through which academic information is published and consumed inside the same platform students use to organise their time.

Societies and announcements. Students can browse and join societies through the Societies section, while society organisers can manage announcements and events through society administration interfaces. Society events can also be synced into a student's calendar. This supports both consolidation and communication: extracurricular activity is surfaced in the same environment as academic planning, and organisers gain a direct channel to their members.

Group work. The Group Work area allows students to manage shared projects, assign tasks, record member availability, and suggest meeting times. This feature extends the platform from individual organisation into small-scale collaboration, reflecting the reality that much student work is coordinated across groups rather than completed alone.

Analytics, AI support, and campus map. The Analytics page presents study history

and progress information, while the AI study coach provides contextual feedback based on recorded activity. The Campus Map page supports route planning to KCL campuses using the student's saved home address. These features broaden the system beyond basic record-keeping by helping students interpret their behaviour and plan around the physical realities of attending university.

Alongside these functional requirements, the project was shaped by several non-functional expectations. Usability was a priority throughout, as a platform intended to reduce cognitive load must not itself become difficult to use. Security was also essential, because the application stores personal, academic, and role-sensitive data. Finally, the system needed to remain accessible across desktop and mobile form factors so that students could consult and update it throughout the day rather than only when seated at a computer.

Chapter 3

Design

Cate Tay (Cate) Chew, Kausar Khan, Dian Mahmood

3.1 System Overview

StudyFlax is a web-based time management application built for students and teaching staff at King's College London. King's College London students typically manage their timetable, assignment deadlines, task lists, and society activities across several separate tools, and keeping track of all of them at once is difficult. The platform brings all of that into a single interface. Teaching staff can also use it to create and manage modules, set deadlines, and publish reading lists, which students then see directly within their own dashboard.

The system is split into two independently deployed parts: a React frontend served via Vite, and a Django backend connected to a PostgreSQL database. The two sides communicate only through a RESTful HTTP API, so neither is coupled to the other's implementation. The backend also connects to several external services: the King's College London timetabling system for importing student schedules, OpenAI for the AI study coach, Mapbox for the campus map, and the Transport for London API for public transport routing. The development environment is containerised using Nix Flakes and Docker Compose, which kept the setup consistent across the whole team.

3.2 Architectural Pattern

The system follows a three-tier architecture, separating the user interface, application logic, and data storage into distinct layers (Figure 3.1). A decoupled approach, in which the frontend and backend are independently deployed and communicate only through a REST API, was preferred over server-side rendering with Django templates. The application requires a high degree of interactivity: the calendar supports drag-and-drop rescheduling and live event resizing, the dashboard widgets update independently of one another, the study timer runs continuously in the background, and several views maintain complex local state across user interactions. Rendering these interfaces through server-generated templates would require a full round-trip to the server for every meaningful user action, which is neither fast enough nor architecturally practical for this kind of feature

set. A dedicated React SPA handles its own state and re-renders incrementally without involving the server, while the Django backend focuses entirely on business logic and data persistence.

The presentation tier is the React SPA running in the user's browser. It has no direct access to the database and holds no session state shared with the backend. All data is fetched and submitted through HTTP requests to the REST API, and the application re-renders based on the responses it receives.

The application tier is a Django REST Framework server responsible for all business logic. It exposes a single API namespace at `/api/`, with endpoints organised by feature area, for example `/api/events/`, `/api/tasks/`, and `/api/societies/`. Authentication is handled per request: each request carries a token in the `Authorization` header, which the server validates using DRF's `TokenAuthentication` class. No session state is held server-side, keeping the application tier fully stateless.

The data tier is a PostgreSQL database accessed exclusively through Django's ORM. No raw SQL is used anywhere in the codebase. In development, user-uploaded files such as profile photos are written to disk and served through a configured media root.

External services (OpenAI, Mapbox, and the TfL API) are called from the application tier only, never from the frontend. This ensures that API keys remain server-side and that the backend can validate and transform external responses before returning them to the client.

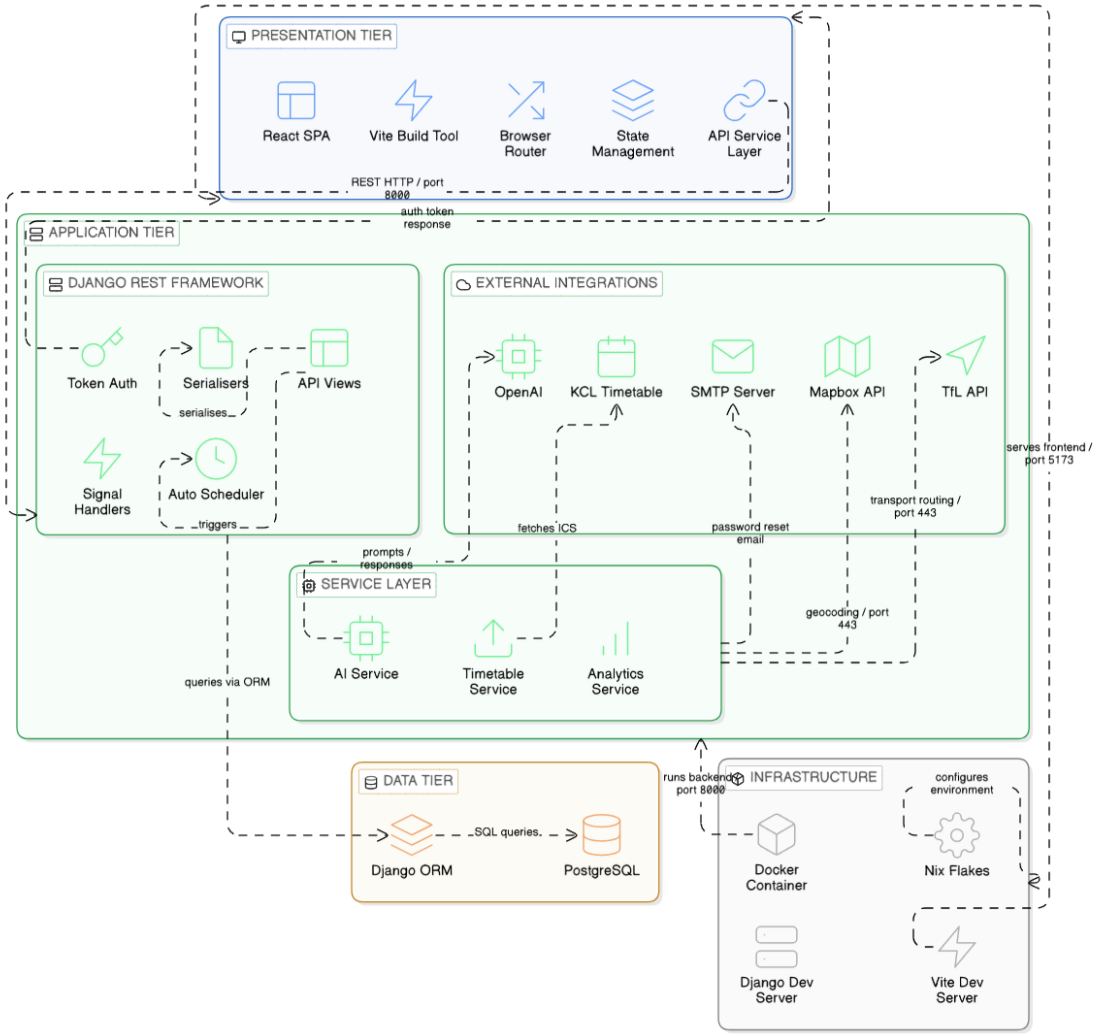


Figure 3.1: Three-tier architecture of StudyFlax.

3.3 Frontend Design

The frontend is built using React with Vite as the build tool, and follows a component-based architecture. The application is structured around a set of page-level components and over 180 smaller reusable components, organised by feature domain (e.g. `calendar/`, `tasks/`, `societies/`).

Routing is handled by React Router, with all authenticated pages wrapped in a shared `DashboardLayout` component that provides the navigation bar and global UI shell. This ensures consistent layout and navigation behaviour across the application without duplicating structure in each page.

State management follows two patterns depending on complexity. Simple, localised state is handled with React's built-in `useState` and `useEffect` hooks. For more complex, cross-component state, the application uses React Context combined with a reducer pattern. The most notable example is `CalendarContext`, which manages event data, modal visibility, and selected date state across the entire calendar feature using a dedicated reducer for predictability. Nine context providers are used in total, covering areas such as theming, notifications, task completion, and timer state.

API communication is abstracted into a dedicated `api/` service layer, keeping data-fetching logic separate from component rendering logic. Custom hooks (e.g. `useCalendarEvents`, `useTimerLogic`) further encapsulate reusable behaviour and are consumed by page components.

Styling uses a combination of CSS Modules and styled-components, with all colour values defined in a central `variables.css` file. This is described further in Section 3.7.

3.4 Backend Design

The backend is a single Django project (`Flax10`) with one application (`studyFlax`). All models, serialisers, views, and business logic sit within this app, with each feature domain organised into its own subdirectory.

3.4.1 API Layer

All API endpoints are registered under `/api/` and routed through a single URL configuration at `studyFlax/urls/api_urls.py`. Most endpoints use plain function-based views with `@api_view` decorators, which kept things simple for endpoints that do not need the full CRUD pattern. Two DRF `ViewSet`s are registered via a `DefaultRouter`, specifically for study session history and calendar sharing, where the router-generated routes were a better fit.

The API covers the following functional domains: authentication, user profiles and settings, calendar events and timetable import, modules and deadlines, tasks and task lists, study timer sessions, group work projects, societies, notifications, analytics, and admin tools.

3.4.2 Models

Models are split into domain subdirectories under `studyFlax/models/`: `auth/`, `calendar/`, `modules/`, `tasks/`, `timer/`, `society/`, `groupwork/`, `social/`, and `notifications/`. The `User` model extends Django's `AbstractUser`, with added role fields (`student`, `teacher`, `pending_teacher`), profile fields such as photo, bio, and home address with geocoordinates, and a linked `UserSettings` model for per-user preferences including theme, daily study targets, and sleep schedule. The custom user model was defined before any initial migrations were run, which was important because retrofitting it afterwards would have required a full database reset.

3.4.3 Serialisers

Each model domain has a corresponding serialiser module under `studyFlax/serialisers/`. DRF `ModelSerializer` is used throughout. Where the API response needs to differ from the raw model fields, custom `validate_*` methods or `SerializerMethodField`s are added, for example to resolve nested relationships or compute derived values such as reading time estimates.

3.4.4 Signals

Django signals handle side effects that should not be mixed into view logic. When a group work suggestion is responded to, or a society announcement is published, a signal handler creates the relevant `Notification` record automatically. Notifications are therefore created reliably regardless of what triggered the change.

3.4.5 Authentication

Registration creates a `User` record and returns a DRF auth token in the same response. Login validates credentials and returns the existing token. Logout deletes the token server-side so it is immediately invalidated. Password reset uses a time-limited token stored in the `ResetPasswordToken` model, with the reset link sent by email through Django's SMTP backend.

3.4.6 Admin

The backend exposes a set of admin-only API endpoints under `/api/admin/`, restricted to users with the `staff` role. These allow administrators to manage user accounts, oversee registered societies, and access platform-wide analytics without going through Django's built-in `/admin/` panel. Django's default `/admin/` interface is also available for direct database-level inspection and management during development. Role-based access is enforced at the view level using a custom permission check against the `is_staff` flag on the `User` model.

3.5 UML Component Diagrams

The system's architecture is illustrated across the two component diagrams below. Figure 3.2 shows the frontend architecture, including the browser routing layer, state management contexts, page components, and API service layer. Figure 3.3 observes the backend architecture, including the Django REST Framework views, signal handlers, serialisers, ORM layer, PostgreSQL database, and external service integrations.

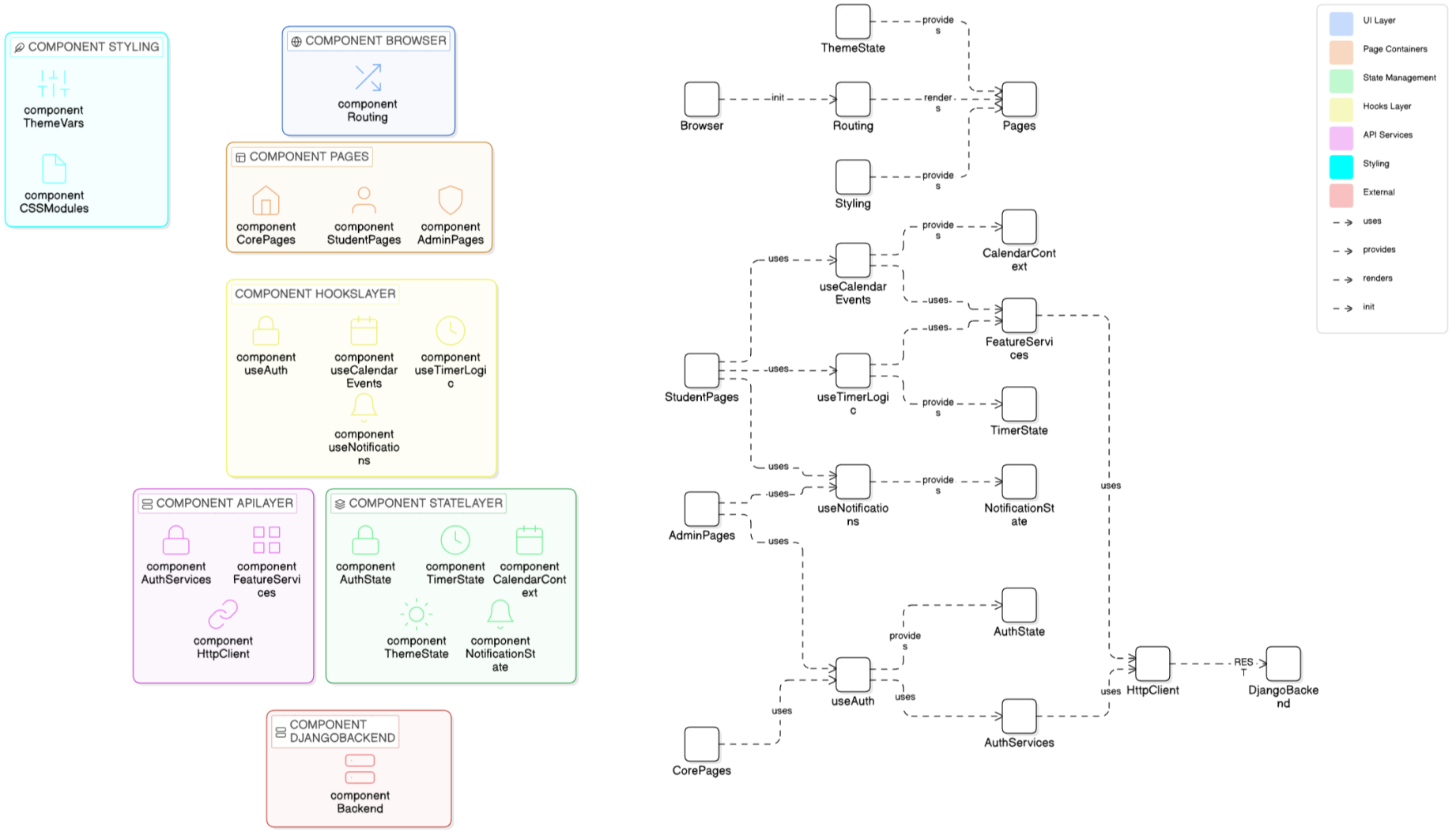


Figure 3.2: Component Diagram — Frontend Architecture

Frontend Design Analysis

High Cohesion:
The frontend demonstrates high cohesion through a careful separation of responsibilities across its architectural layers, as illustrated in Figure 3.2. Each module encapsulates a single, well-defined concern within the system: page-level components are responsible only for UI composition, shared state is centralised within dedicated contexts such as `CalendarContext` and `AuthState`, and domain-specific custom hooks encapsulate reusable behaviour independently of UI components. As a concrete example, `useCalendarEvents` encapsulates all calendar data-fetching behaviour and exposes it to `StudentPages` without those pages requiring any knowledge of the underlying API calls or state management implementation. This organised approach is consistently applied across all feature areas including calendar, tasks, timer, and notifications, ensuring that related logic and presentation are grouped together within their respective modules. Each layer therefore has a well-defined purpose, improving readability, maintainability, and testability across the codebase.

Low Coupling:
The frontend enforces low coupling through the implementation of clear interaction boundaries between components and layers. Communication between modules occurs through well-defined interfaces: props for UI components, context providers for shared state, and service abstractions within the `APILayer` for backend communication. By extracting domain logic into custom hooks and delegating data access to service modules, components avoid direct dependencies on the implementation details of other parts of the system. For example, page components depend on context and hooks rather than directly invoking API calls, reducing tight coupling between the UI and data-fetching logic. The `HttpClient` serves as the single point of communication between the frontend and the Django backend, ensuring that changes to the backend interface require updates only within the `APILayer` rather than propagating throughout the component tree. This separation ensures that changes within a given module have minimal impact on other modules, provided that external interfaces remain stable, reinforcing modularity, scalability, and ease of refactoring across the frontend codebase.

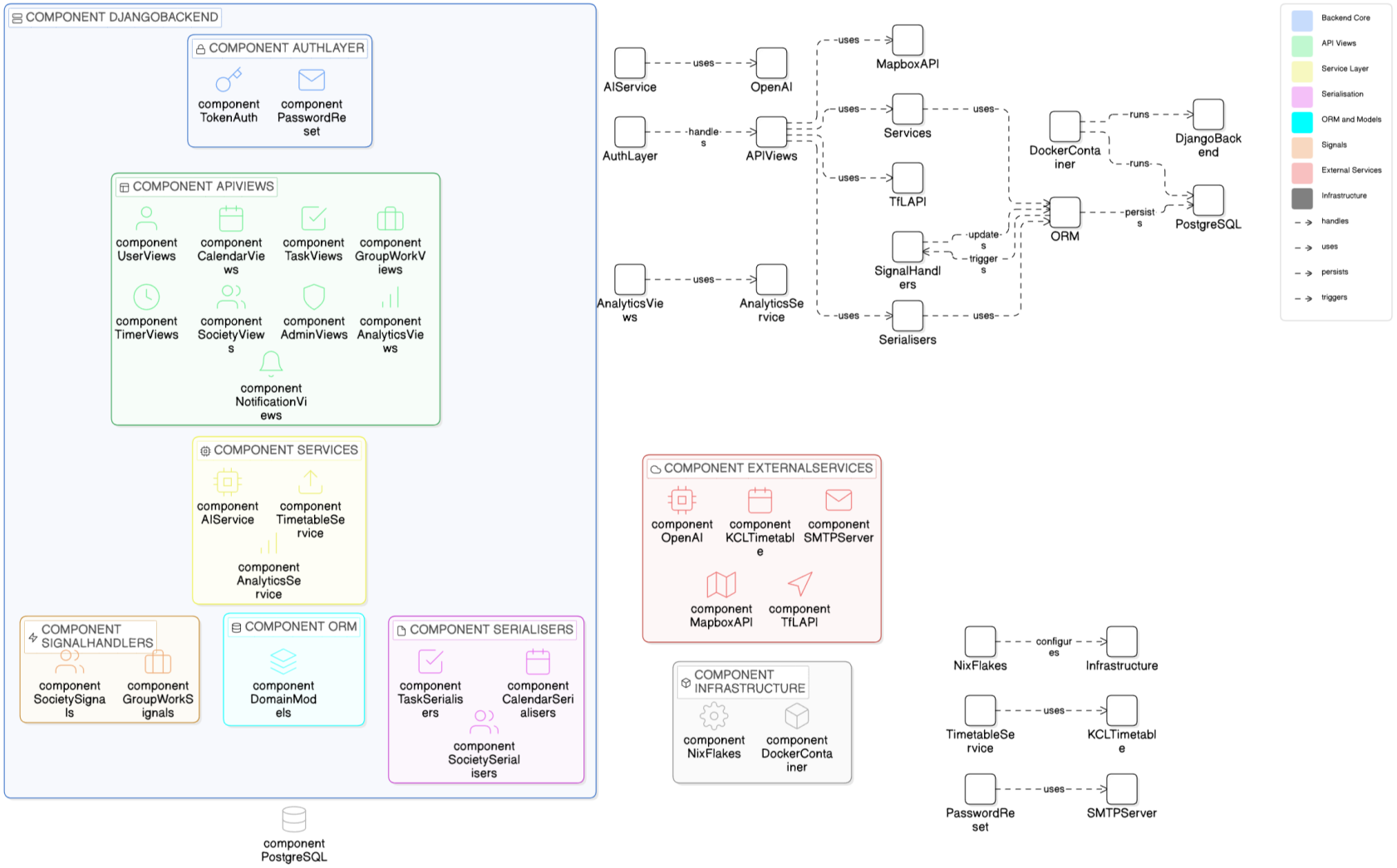


Figure 3.3: Component Diagram — Backend & Infrastructure Architecture

Backend Design Analysis

High Cohesion:
The backend also observes high cohesion by grouping related responsibilities in clearly defined layers and feature-oriented subpackages, which can be seen in Figure 3.3. Within the `studyFlax` application, domain models are organised under specific subdirectories such as `auth`, `calendar`, `modules`, `tasks`, `society`, `timer`, `notifications`, and `groupwork`, whereas additional concerns such as serialisers, views, URLs, signals, and utilities are stored and maintained in their own dedicated modules. As a result, each backend file or package serves one principal purpose: persisting domain state, validating API data, transforming API data, handling request logic, or wiring routes. To provide an example, the `SignalHandlers` layer is exclusively responsible for event-driven side effects such as creating notifications when a society announcement is published or a group work suggestion is replied to, keeping the logic separate from view and serialiser code. This organisation supports high cohesion because code that changes for the same reason is kept together, making the backend easier to navigate and extend without blurring the responsibilities of individual components.

Low Coupling:
The backend enforces low coupling by mediating interactions between parts of the system through interfaces, instead of having tightly coupled implementations. This is visible in the structure of Figure 3.3. URL configuration delegates incoming requests to specific view modules; these views then rely on serialisers and the ORM for validation and persistence, whereas additional concerns such as creating notifications, handling emails and external service communications are extracted into dedicated signals and service modules, avoiding duplication across request handlers. As an example, the `ORM` layer serves as the single point of database access for both the `Serialisers` and `Services` layers independently, meaning neither layer is aware of the other's existence or implementation. Moreover, the feature-based package structure reduces unnecessary cross-domain dependencies, since calendar logic, task logic, society logic, and authentication logic are each strongly encapsulated within their own backend areas. This ensures that modifications within one domain have minimal impact on other parts of the system, provided that external interfaces remain unchanged. Such a structural approach encourages separation of concerns, maintainability, and ease of independent testing across the backend codebase.

3.6 Entity Relationship Diagrams

This section explores a system data model represented using a set of Entity Relationship (ER) diagrams. The diagrams illustrate how data are structured, stored, and related across the system.

The data model has been decomposed into three focused sub-diagrams to facilitate both comprehension and readability of each functionality.

Figure 3.4 explores the core user and academic data model, delving into models representative of users, modules, and associated academic entities. Figure 3.5 observes functionalities involving productivity and scheduling, with features such as calendar events and study sessions. Figure 3.6 illustrates the social aspect of the system, observing the presence of societies, group projects, and meeting coordination.

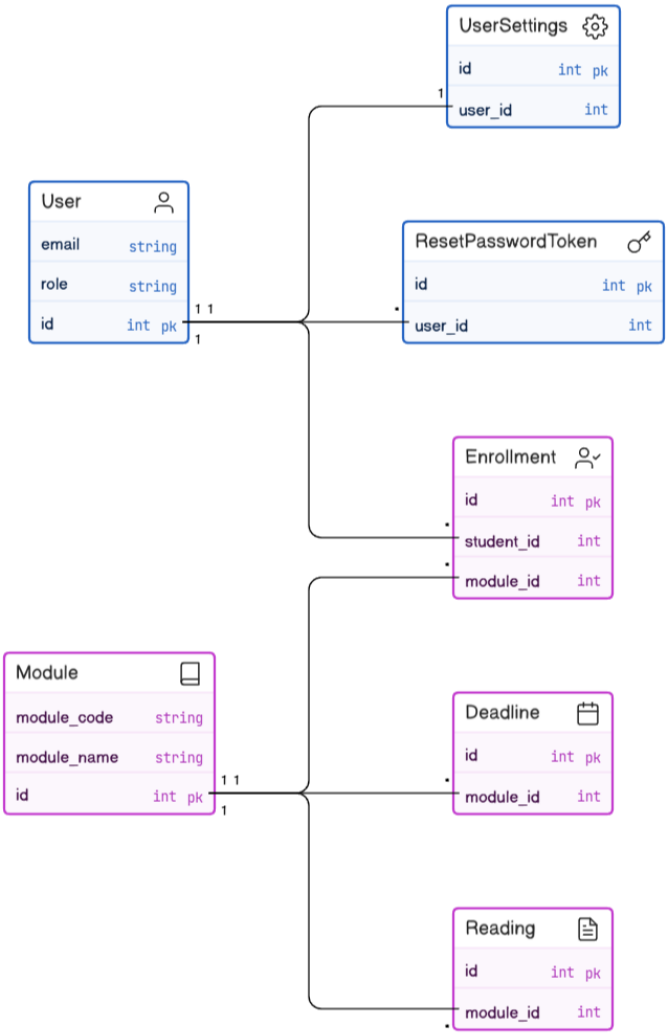


Figure 3.4: ER Diagram - User and Academics

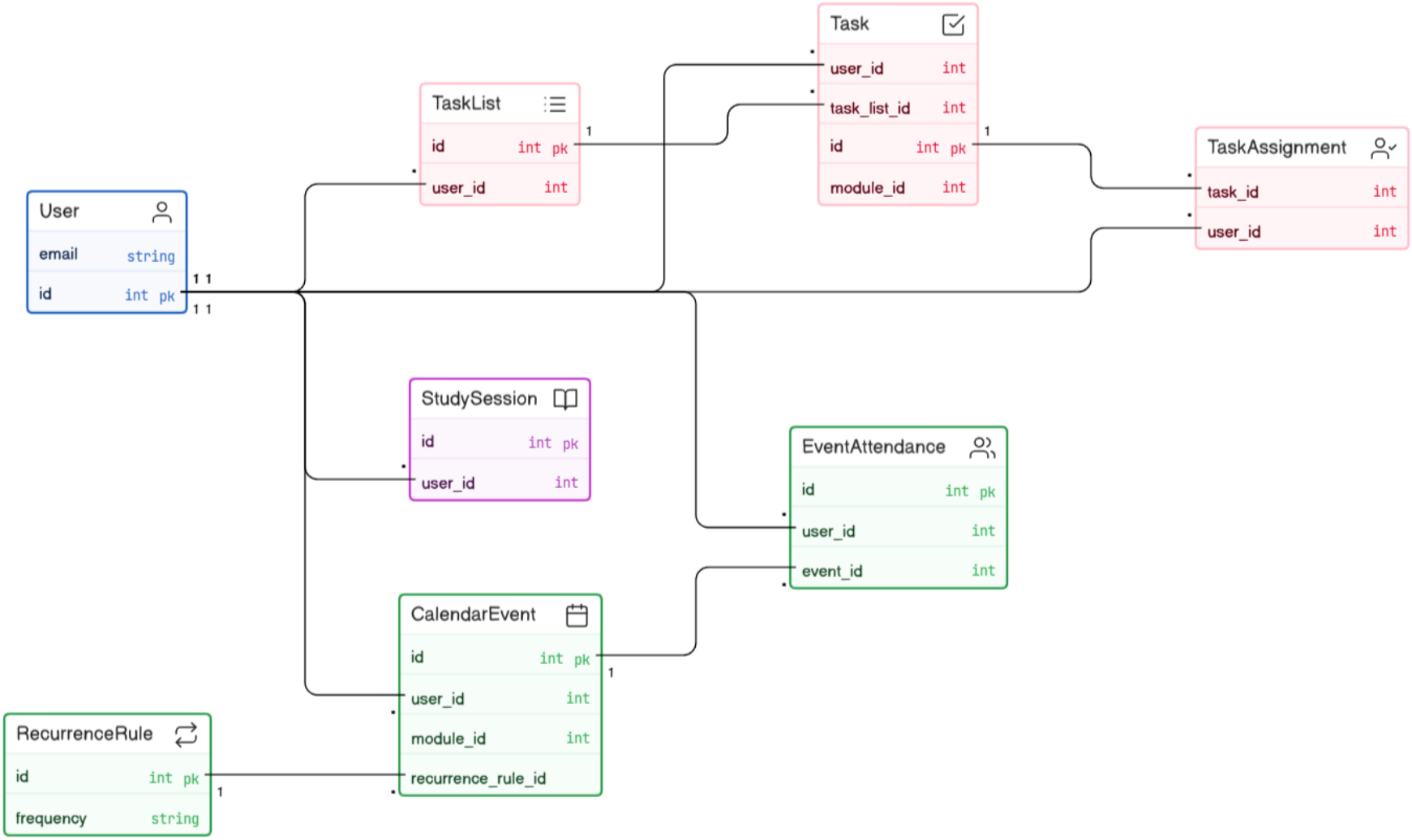


Figure 3.5: ER Diagram - Productivity and Scheduling

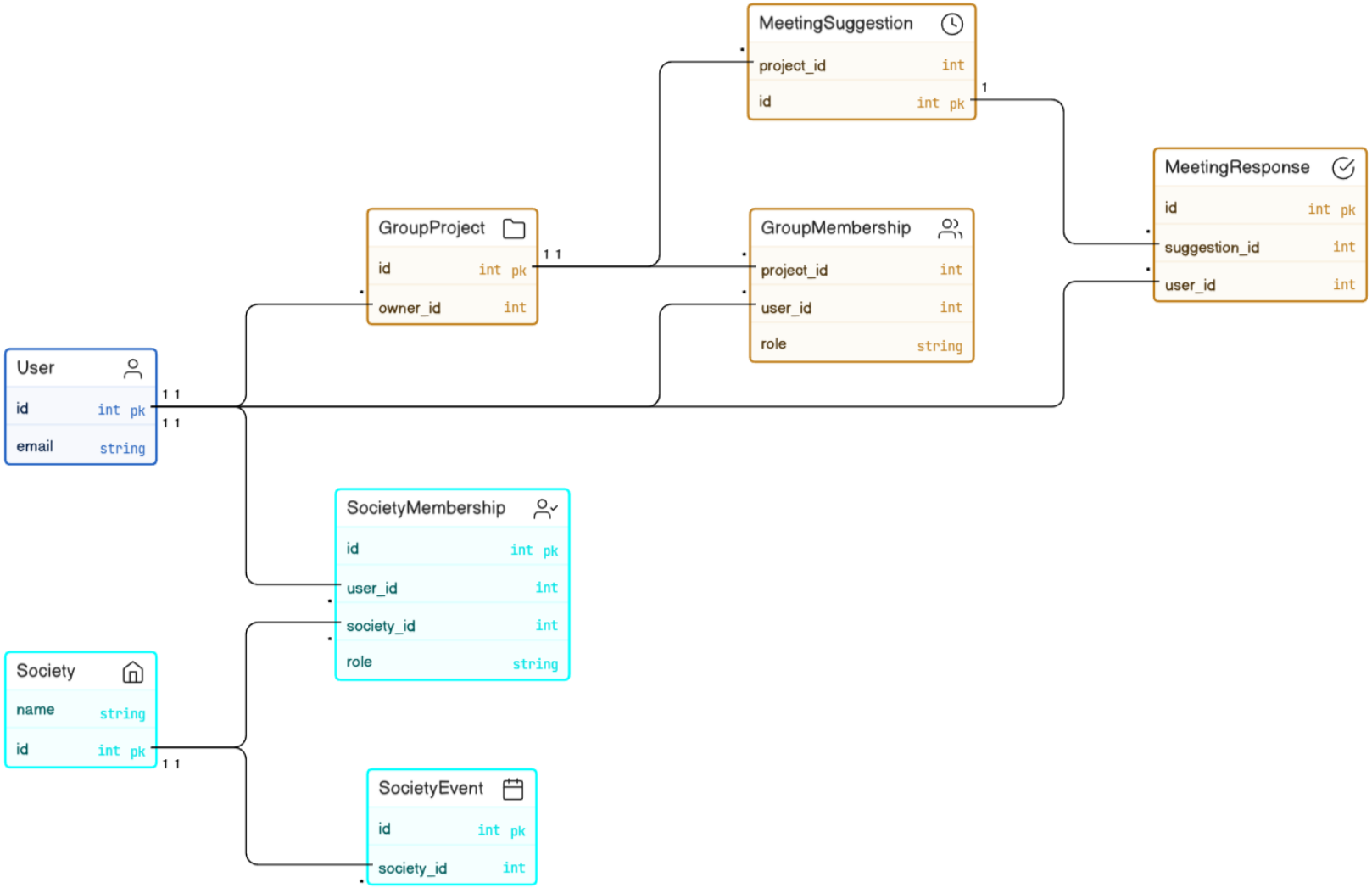


Figure 3.6: ER Diagram - Social and Collaboration

3.7 Key Design Decisions

This section documents the significant design decisions made during development, focusing on choices where alternatives were considered or where the approach was non-obvious.

3.7.1 Custom User Model with Unified Role System

A custom user model was defined from the outset of the project, extending Django's `AbstractUser` to incorporate roles (`student`, `teacher`, and `staff`) and profile-specific fields such as profile photo and home address, with study preferences such as year and major stored in a linked `UserSettings` model. This decision was made early and deliberately, as retrofitting a custom user model after initial migrations would have required resetting the database entirely. The unified model ensures that role-based behaviour, such as restricting module creation to teachers and administration tools to admins, is handled consistently through a single identity model rather than through separate profile tables.

3.7.2 Token-Based Authentication

Token-based authentication was chosen over session-based authentication to keep the backend stateless. No session data is stored on the server, and each request is authenticated independently via a token passed in the `Authorization` header. This fits the decoupled SPA and API architecture naturally, as the React frontend and Django REST Framework backend share no server-side state.

DRF's built-in token model was preferred over JSON Web Tokens (JWT), which is a common alternative for single-page applications. JWT is stateless by design: once issued, a token cannot be invalidated without additional infrastructure such as a server-side denylist or a short-expiry and refresh-token cycle. DRF tokens, by contrast, are stored in the database and are deleted immediately when a user logs out, providing instant and unconditional invalidation. For a student-facing application where users may share devices, reliable session termination on logout was considered more important than the marginal infrastructure simplicity that JWT statelessness provides.

3.7.3 Course-Prefix Mapping for Module Onboarding

During onboarding, modules are automatically assigned to a student based on their selected major using a course-prefix mapping (e.g. selecting Computer Science maps to all modules with the prefix `6CCS` or `5CCS`). The alternative of manual per-user module assignment was rejected as it would not scale and would place an unnecessary burden on both users and administrators. The prefix-mapping approach means that as new modules are added to the system, they are automatically surfaced to the correct students without any additional configuration.

3.7.4 CSS Variables for Theming

Colour values are primarily defined as CSS custom properties in a central `variables.css` file, which underpins the light and dark theme toggle and enforces visual consistency

across the UI.

3.7.5 Tasks as a Standalone Page

The task management feature was designed to function as a central point of integration across the application: tasks can be linked to calendar events, completed through study timer sessions, and associated with module deadlines. Supporting these relationships requires the tasks view to offer dedicated navigation, filtering by module or deadline, and persistent interaction state. A floating overlay panel does not provide the space or the navigational context these integrations require, and embedding cross-feature links within an overlay would create a confusing interaction model where one part of the interface triggers actions in another. Implementing tasks as a dedicated standalone page gives the feature the same structural status as the calendar, modules, and other core sections of the application, and makes it possible to build the full set of cross-feature interactions without imposing constraints on any surrounding page.

3.7.6 Navigation: Top Bar and Bottom Bar

The navigation system needed to provide persistent access to all major sections of the application without affecting the layout of page content. A collapsible sidebar does not satisfy this requirement: toggling it changes the width available to the main content area, which repositions elements on pages with dense or precisely laid-out content such as the calendar, the Bento dashboard, and the campus map. A permanently open sidebar resolves the layout-shift problem but consumes a significant portion of the viewport on smaller screens, where horizontal space is already limited.

A persistent top navigation bar on desktop and a fixed bottom navigation bar on mobile address both constraints. Neither bar occupies any horizontal space used by page content, so layouts remain stable regardless of navigation state. The bottom bar on mobile follows the established platform convention for primary navigation and places the most frequently used sections within thumb reach. This combination gives users constant access to navigation while ensuring that page content is never displaced.

3.7.7 Modular Bento Dashboard with Widget Customisation

Rather than a static homepage layout, the dashboard was implemented as a modular Bento-style grid in which users can personalise their view by selecting which widgets to display. Users can choose from widgets including a daily timeline, upcoming deadlines, task list, reading plan, and AI study coach panel. Widget preferences are persisted in the user's settings. This design decision prioritises personalisation: different students have different workflows, and a fixed layout would not serve all use cases equally. The grid structure ensures visual consistency is maintained regardless of which widgets are active.

3.7.8 Auto-Scheduler Design

The auto-scheduler feature generates study events based on a student's module deadlines, daily study time preferences set in settings, and sleep schedule (no events are scheduled during the user's configured sleep hours). Rather than implementing the scheduler as a

standalone module with its own event model, a wrapper class was used to reference and manage the generated schedule events. This keeps the scheduler coupled to the existing calendar infrastructure, meaning future changes to calendar functionality automatically apply to scheduled events without requiring parallel updates to a separate module. Schedules are named and managed through a dedicated auto-scheduler page, where students can rename or delete a schedule (which removes all associated events). A replan button on the homepage mini-timeline also triggers the auto-scheduler to update the current schedule based on the latest deadlines and preferences.

3.8 External Integrations

The system integrates with four external services.

iCalendar (RFC 5545) format is supported for timetable import. King's College London students can provide their timetable URL, taken from the King's College London timetable website, and the system downloads and parses the `.ics` file to extract scheduled classes and import them directly into the application's calendar as events.

OpenAI API powers the AI study coach feature. When a student requests insights or asks a question through the coach interface, the system constructs a context object from the student's study session history, streak statistics, peak study hours, dark mode usage patterns, and upcoming calendar events, and sends this to the OpenAI API. The response is returned as personalised, data-aware feedback. The integration is conditionally loaded: if the `openai` package is not installed or the API key is not configured in the environment, the backend returns an HTTP 503 response and the coach panel renders an inline error message in place of the coach output, leaving all other features of the application entirely unaffected.

Mapbox GL is used to render the campus map feature, which displays the five King's College London campus locations (Strand, Guy's, Waterloo, St Thomas, and Denmark Hill) and provides journey planning functionality. King's College London students can calculate walking routes to any campus using the Mapbox Directions API, or public transport routes using the TfL Journey Planner API, based on their saved home address.

Transport for London (TfL) API is used alongside Mapbox to provide public transport routing on the campus map, giving King's College London students the option to plan their journey to campus by tube, bus, or rail.

Chapter 4

Testing

Zerun (Harry) Chen, Hamed Latif

Testing was integrated throughout development to ensure the reliability and correctness of the system. The application consists of a backend implemented using Python and Django, and a frontend implemented using React with Vite. Automated tests were developed for both parts of the system and were complemented by manual testing of the deployed application. The team followed an **Agile development approach with continuous testing**, meaning tests were written alongside feature development.

4.1 Automated Testing

Backend tests were implemented using Django's built-in testing framework together with the Python coverage library. These tests validate database models, helper functions, and API endpoints. For example, tests verify behaviour for models such as **test_deadline_model.py**, which checks default values, valid deadline types, and cascading deletion with its parent module, and **test_notification_model.py**, which verifies notification-related model behaviour. API endpoints are also tested in files such as **test_auth_login_view.py**, which checks successful and unsuccessful login responses, **test_task_detail_view.py**, which validates task retrieval and update behaviour, and **test_society_view.py**, which covers society-related endpoint behaviour. A representative backend test verifying model behaviour is **test_deadline_model.py**, because it confirms that the model stores required data correctly and applies expected defaults when optional fields are omitted.

Frontend tests were written using **Vitest**, which integrates with the Vite development environment. These tests validate React components, hooks, context providers, and full page behaviour. Examples include component tests such as **TaskCard.test.jsx**, which checks interactions such as opening menus, editing tasks, and starting study sessions, **NotificationToast.test.jsx**, which verifies toast rendering and timed dismissal behaviour, and **UserSearch.test.jsx**, which validates filtering and result selection. Hook tests such as **useTimerLogic.test.js** and **useCalendarEvents.test.js** verify state updates and asynchronous event loading. A representative frontend component test verifying user interaction is **TaskCard.test.jsx**, because it exercises several user actions and confirms that the interface responds correctly.

The frontend test suite contains **961 test files across 157 directories**, while the back-

end suite contains **100 test files across 22 directories**. Together this results in **1,061 automated test files**. After implementing each feature, the developer responsible for that feature also wrote the corresponding automated tests, ensuring that new functionality was validated immediately and that each team member contributed automated tests (II.5).

4.2 Test Coverage

Code coverage tools were used to evaluate the completeness of the automated tests. During **week 6**, the team reviewed the entire test suite and improved it until coverage reached **100%**. This ensured that all major components of the system were executed during automated testing and satisfied the coverage requirements in the marking criteria (III.5.1, III.5.2, IV.5.1, IV.5.2).

Tests were designed to cover multiple input partitions, including **valid inputs, invalid inputs, and edge cases**. For example, tests validate behaviour when requests contain missing fields or invalid authentication tokens. An example input validation test can be found in **test_auth_login_view.py**, where invalid credentials return **401 Unauthorized** and empty login data returns **400 Bad Request**, demonstrating that incorrect inputs are handled separately from valid requests (V.5.1, V.5.2).

4.3 Test Code Quality

Test code quality was maintained through the use of reusable fixtures and mixins to avoid duplicated setup logic. Shared configuration is defined in **test_case_support.py**, which provides reusable base classes, authenticated API client setup, and helper assertions for event-related tests. Additionally, a script was used to detect duplicated code across the repository, ensuring repeated test implementations were refactored or removed (V.3.6).

Another script checked the structure of test functions to ensure they remained short and readable. Tests were typically limited to **fewer than 25 lines** and avoided deep control-flow nesting. A concise unit test verifying a single behaviour can be found in **test_seed_homepage_widgets.py**, where one short test confirms that only the expected core homepage widgets are selected during seeding. Tests also follow clear naming conventions such as **TaskCard.test.jsx** and **test_task_detail_view.py** (V.3.5, V.3.7).

4.4 Manual Testing

Manual testing was performed during **weekly team meetings**. The team tested both the development and deployed versions of the system by performing common user workflows such as creating tasks, navigating the calendar, and interacting with social features.

Manual testing was primarily used to verify **user interface behaviour and usability**, which are difficult to fully validate using automated tests. A detailed list of manual tests, including actions and expected results, is provided in **Appendix A**. This process also ensured the deployed application was regularly validated (III.4.2).

4.5 Limits of Testing

Automated testing provides advantages such as **speed, repeatability, and regression detection**. However, automated tests cannot easily detect issues related to layout or overall user experience. Manual testing helps identify such issues but cannot be executed as consistently as automated tests.

4.6 Threats to Validity

Although the project includes **1,061 automated test files** and achieves **100% code coverage**, this does not guarantee that the system is completely free of defects. Code coverage indicates that code was executed during testing, but it does not ensure that every possible logical error or edge case has been detected.

Some tests also rely on simulated inputs rather than real-world conditions. For example, **test_email_utils.py** validates behaviour using mocked email-sending and template-rendering functions rather than a real mail server, which makes the test reliable and isolated but less representative of full production behaviour.

Despite these limitations, the combination of extensive automated testing, full coverage analysis, and regular manual testing provides strong confidence in the reliability of the system.

Chapter 5

Project management

Eric Almeida

This chapter reflects on the management strategies employed throughout the project lifecycle. The team adopted an Agile-inspired approach, utilising a Kanban workflow to manage the transition from a client-led project to an academic one.

5.1 Team Structure and Roles

To ensure accountability and specialisation, each team member was assigned a specific leadership role alongside their development duties (see team feedback, meeting of 20 January 2026):

- **Hamed Latif**: Client Liaison and Final Submission Manager
- **Kausar Khan**: Trello Board Manager
- **Eric Almeida**: User Story and Task Manager
- **Aishah Ahmed**: LaTeX Report Manager
- **Cate Tay (Cate) Chew**: GitHub and Version Control Manager
- **Zerun (Harry) Chen**: Deployment Manager
- **Dian Mahmood**: Code Quality Manager
- **Indixora Jozal**: People's Engagement Manager
- **Binuli Nevanma Markawitage**: Meeting Setup Manager

5.2 Project Evolution

The project initially began as a client-facing application for *MakCorp*, a platform managing data for over 800 ASX-listed companies, with the team tasked with building a data warehouse focused on analysing mining stocks. At the start of week three, after careful consideration, the team collectively decided to pivot to an academic project: **StudyFlax**, a time management system for King's College London students and teaching staff.

The decision was not taken lightly. The team recognised the value of working with a live production system and appreciated the willingness of MakCorp to collaborate with students. However, given the 10-week delivery timeline and the module's emphasis on structured development, testing, and version control, the prevailing uncertainty around system access, development environments, and technical architecture introduced risks that could not be confidently managed within the academic constraints of the project. The pivot was therefore a matter of aligning project scope with what the team could deliver reliably within the semester, rather than a reflection of MakCorp's engagement or the quality of the opportunity.

To mitigate the risk of reduced development time following this switch, the team increased its target goal to 18 user stories per week. This allowed the team to regain momentum and ensure all core features of StudyFlax were delivered on schedule.

5.3 Communication and Collaboration

The team's communication strategy relied on a combination of synchronous and asynchronous tools:

- **Meetings**: The team first decided to hold two formal meetings per week (Mondays at 3:00 PM and Fridays at 2:00 PM) to discuss progress and blockers. However, due to unforeseen commitments from team members, we decided to schedule team meetings 2 days before each meeting to accommodate those who had flexible schedules.
- **Instant Messaging**: WhatsApp was used for daily coordination, with a protocol requiring members to reply within the same day.
- **Task Tracking**: A Trello Kanban board was used to visualise the backlog, in-progress tasks, and completed features (see team feedback, Trello Kanban board).

5.4 Software Quality and Standards

To maintain a high standard of software engineering, the team enforced strict coding conventions:

- **Code Reviews**: All pull requests required approval from at least three team members before merging into the main branch.
- **Function Length**: Methods were limited to a maximum of 20-25 lines to ensure readability and high cohesion.
- **Testing**: The team aimed for 100% code coverage, with testing methods capped at 25 lines.

5.5 Reflection and Lessons Learned

The most significant lesson learnt was the importance of early risk identification. Although the initial communication issues with MakCorp were a setback, communication within the team and a clear division of responsibilities ensured that the team remained cohesive

during the transition. The use of a decentralised leadership structure allowed the team to move faster than a traditional hierarchy would have allowed. For future projects, the team would establish a more robust contingency plan for client-side delays earlier in the process.